Which resources and inputs does a cost model for Apache Wayang operators need?

A measurement-based analysis for the Java, Spark and PostgreSQL platforms

1 Summary

A cost model for a Wayang operator predicts how much of each resource the operator uses: CPU time, memory, garbage collection (GC), disk I/O, data moved between processes or machines, and a fixed overhead per job. Every extra resource is another prediction target and another source of error, so we measured which ones actually matter. For each platform and operator we ran the operator inside a plan and subtracted the same plan without it, so every number below is what the operator itself adds. A resource is **needed** when it is clearly above measurement noise and large relative to the operator's total cost; it is **not needed** when it is negligible for every operator of the platform.

Java

- **Needed:** CPU (FlatMap, ReduceBy, GroupBy, Sort, Join, Cartesian, TextFileSink); disk I/O (TextFileSink).
- Only in some configurations: fixed per-job overhead (TextFileSink).
- **Not needed** (negligible for every operator): garbage collection.
- *Not assessed in this data* (not measured): memory.
- Operators whose own cost is below measurement noise: Map, Filter, GlobalReduce, Count, UnionAll (a constant or a per-row cost from the source/sink is enough).

Spark (single node)

- **Needed:** CPU (Map, FlatMap, ReduceBy, GroupBy, Distinct, Sort, Join, Cartesian); memory (GroupBy, Sort, Join); disk I/O (TextFileSink); data movement (GroupBy, Sort, Join); fixed per-job overhead (ReduceBy, Distinct, Sort, TextFileSink).
- **Not needed** (negligible for every operator): garbage collection.
- Operators whose own cost is below measurement noise: Filter, Count, UnionAll (a constant or a per-row cost from the source/sink is enough).

Pending: Spark (cluster), PostgreSQL are not in the data used for this version of the report.

2 Method

Measurement. Each configuration (operator, input size, number of distinct keys, row width, selectivity, JVM and platform settings) runs in a fresh JVM. It executes two plans alternately after warm-up runs: a *baseline* (source → parse → sink) and the *operator plan* (the same with the operator inserted). The operator's cost is the difference of the two runs of a pair, in every metric;

a configuration has 3–7 pairs. Metrics are read from the JVM and the operating system over the window from the first record to the end of the job, so Wayang's own optimisation is excluded. Pairs with a run measured at a reduced CPU clock or next to other busy processes are flagged by the profiler and left out here.

Resources and the columns they come from.

Platform	Resource	Measured column
Java	CPU	thread_cpu_exec_ns
	Memory	mem_held_bytes
	GC	gc_time_exec_ms
	Disk I/O	os_disk_read/write_bytes
Spark (single node)	CPU	task_cpu_ns
	Memory	task_peak_execution_memory_bytes_max × task_slots
	GC	task_gc_ms
	Disk I/O	spill_disk_bytes + output_bytes + os_disk_read_bytes
	Data movement	shuffle_write_bytes

The *fixed overhead* is not a column: runtime is fitted as fixed + per-unit × (input rows + output rows) through the medians of the smallest and the largest 20% of the configurations of the operator, and the fixed part is compared with the median added runtime.

Rating. For every resource and operator we compute (i) the *signal*: the share of configurations whose median added amount is above the spread between pairs (IQR) and above a floor (1 ms, 64 KiB), and (ii) the *magnitude* in the median configuration: for CPU and GC the share of the operator's added runtime (CPU time / (wall time × threads)); for memory, disk and data movement bytes per input byte; for the fixed overhead its share of the median added runtime. ● *important*: signal $\geq$ 50% and magnitude $\geq$ 0.30 (time) or 0.50 (bytes); ◐ *sometimes*: signal $\geq$ 20% and magnitude $\geq$ 0.05, or the 90th percentile $\geq$ the important threshold; ○ *negligible*: otherwise. A superscript $-$ means the operator *reduces* the resource compared to its baseline (e.g. a filter sends fewer rows onward). A dot means the operator's added runtime is below noise in more than 80% of its configurations, so no rating is meaningful.

Input properties. For every operator we compute the Spearman rank correlation ρ between its added runtime and each property of the configuration (input rows, output rows, distinct keys, row width, selectivity, left×right rows for binary operators). $|\rho| \geq 0.6$ counts as *needed*, $0.3 \leq |\rho| < 0.6$ as *weak*, below as *not needed*.

Data used

Platform	Configurations	Clean pairs	Disturbed pairs left out
Java	234	702	63
Spark (single node)	210	630	165

Source files: `../java-profiler/results-2026-11-02/pairs.csv`, `../spark-profiler/results/pairs.csv`.

3 Results by platform

3.1 Java

Operator	CPU	Memory	GC	Disk I/O	Data movement	Fixed overhead
Map	·	·	·	·	–	·
Filter	·	·	·	·	–	·
FlatMap	●	–	○	○	–	○
ReduceBy	●	–	○	○	–	○
GroupBy	●	–	○	○	–	○
GlobalReduce	·	·	·	·	–	·
Count	·	·	·	·	–	·
Distinct	◐	–	○	○	–	○
Sort	●	–	○	○	–	○
Join	●	–	○	○	–	○
UnionAll	·	·	·	·	–	·
Cartesian	●	–	○	○	–	○
TextFileSink	●	–	○	●	–	◐

Magnitudes (median configuration; CPU, GC and fixed overhead: share of added runtime, memory, disk, data movement: bytes per input byte).

Operator	Configs	CPU	Memory	GC	Disk I/O	Data movement	Fixed overhead
Map	10				*cost below noise*		
Filter	50				*cost below noise*		
FlatMap	30	1.00	–	0.00	0.00	–	0.02
ReduceBy	26	0.99	–	0.00	0.00	–	–
GroupBy	23	1.00	–	0.00	0.00	–	0.00
GlobalReduce	9				*cost below noise*		
Count	10				*cost below noise*		
Distinct	26	1.00	–	0.00	0.00	–	0.00
Sort	6	1.00	–	0.00	0.00	–	0.10
Join	12	1.00	–	0.00	0.00	–	0.00
UnionAll	10				*cost below noise*		
Cartesian	12	1.00	–	0.00	0.00	–	0.05
TextFileSink	10	1.00	–	0.00	1.00	–	0.12

A dash means not applicable or not measured. "Configs" is the number of configurations (each with 3–7 pairs) behind the operator's row: ratings from a handful of configurations are weak evidence.

Input properties (Spearman ρ with the operator's added runtime; bold: needed; a dash: not applicable, constant in the data, or fewer than 8 configurations).

Operator	input rows	output rows	distinct keys	row width	selectivity	left×right rows
FlatMap	**+0.69**	**+0.96**	–	**+0.85**	–	–
ReduceBy	+0.31	**+0.92**	**+0.90**	**+0.85**	–	–
GroupBy	**+0.93**	+0.50	+0.48	**+0.76**	–	–
Distinct	**+0.61**	**+0.80**	**+0.78**	**+0.90**	–	–
Sort	–	–	–	–	–	–
Join	**+0.86**	**+0.98**	+0.42	**+0.69**	–	**+0.86**
Cartesian	**+0.93**	**+0.96**	–	+0.37	–	**+0.96**
TextFileSink	**+0.98**	**+0.98**	–	**+0.98**	–	–

3.2 Spark (single node)

Operator	CPU	Memory	GC	Disk I/O	Data movement	Fixed overhead
Map	●	○	○	○	○	○
Filter	·	·	·	·	·	·
FlatMap	●	○	○	○	○	○
ReduceBy	●	◐	○	○	○	●
GroupBy	●	●	○	○	●	◐
GlobalReduce	◐	○	○	○	○	○
Count	·	·	·	·	·	·
Distinct	●	◐	○	○	○	●
Sort	●	●	○	○	●	●
Join	●	●	○	○	●	◐
UnionAll	·	·	·	·	·	·
Cartesian	●	○	○	○	○	◐
TextFileSink	◐	○	○	●	○	●

Magnitudes (median configuration; CPU, GC and fixed overhead: share of added runtime, memory, disk, data movement: bytes per input byte).

Operator	Configs	CPU	Memory	GC	Disk I/O	Data movement	Fixed overhead
Map	10	0.41	0.00	0.00	0.00	0.00	–
Filter	50				*cost below noise*		
FlatMap	30	0.50	0.00	0.00	0.00	0.00	0.03
ReduceBy	26	0.34	0.07	0.00	0.00	0.01	0.57
GroupBy	26	0.59	1.15	0.00	0.00	0.82	0.24
GlobalReduce	10	0.19	0.00	0.00	0.00	0.00	–
Count	7				*cost below noise*		
Distinct	13	0.51	0.03	0.00	0.00	0.01	0.61
Sort	5	0.63	1.52	0.00	0.00	0.85	0.38
Join	10	0.89	1.12	0.00	0.00	0.83	0.20
UnionAll	6				*cost below noise*		
Cartesian	12	0.64	0.00	0.00	0.00	0.00	0.23
TextFileSink	5	0.28	0.00	0.00	1.01	0.00	0.51

A dash means not applicable or not measured. "Configs" is the number of configurations (each with 3–7 pairs) behind the operator's row: ratings from a handful of configurations are weak evidence.

Input properties (Spearman ρ with the operator's added runtime; bold: needed; a dash: not applicable, constant in the data, or fewer than 8 configurations).

Operator	input rows	output rows	distinct keys	row width	selectivity	left×right rows
Map	**+0.79**	**+0.79**	–	**+0.79**	–	–
FlatMap	**+0.78**	**+0.97**	–	**+0.79**	–	–
ReduceBy	**+0.60**	**+0.88**	**+0.88**	**+0.97**	–	–
GroupBy	**+0.95**	+0.43	+0.40	**+0.73**	–	–
GlobalReduce	**+0.62**	–	–	**+0.62**	–	–
Distinct	**+0.61**	**+0.86**	**+0.85**	**+0.95**	–	–
Sort	–	–	–	–	–	–
Join	**+0.98**	**+0.90**	**+0.77**	**+0.90**	–	**+0.98**
Cartesian	**+0.96**	**+0.80**	–	**+0.65**	–	**+0.80**
TextFileSink	–	–	–	–	–	–

4 What this means for the cost model

- **Java.** Predict CPU, disk I/O. Garbage collection can be left out or kept as a constant. A per-job fixed cost matters for TextFileSink.

- **Spark (single node).** Predict CPU, memory, disk I/O, data movement, fixed per-job overhead. Garbage collection can be left out or kept as a constant. A per-job fixed cost matters for ReduceBy, GroupBy, Distinct, Sort, Join, Cartesian, TextFileSink.
- **Inputs.** Use the properties marked needed in the matrices above per operator; properties with $|\rho| < 0.3$ for an operator carry no usable signal for it. Output rows is often the strongest driver of filtering and grouping operators, but is unknown before execution: the model should receive Wayang's cardinality estimate for it.

5 Limitations

- On the Java platform an operator runs on one thread, so its CPU time equals its runtime: CPU is then the resource by construction, and the other resources decide only what *else* must be predicted.
- Ratings describe the configurations that were measured (inputs of 10^3–$3 \cdot 10^6$ rows, synthetic data with one row layout); a resource rated negligible here can matter for much larger inputs.
- An operator whose cost is below noise in most configurations is not "unimportant": it is cheap next to reading and parsing its input. Its cost can still add up in long pipelines.
- Data movement for Spark on a single node is the shuffle volume written; only a cluster measures real network traffic between executors.
- Energy and hardware counters (cycles, instructions) exist on macOS only and are not part of this analysis.